UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

University of Canterbury

FINAL EXAM 2024

Prescription Number(s):	**ENCE260-24S2**
Paper Title:	**Computer Systems**

Time allowed: **Two Hours**

Number of pages: **35**

Calculators **ARE** permitted.

ALL Questions **ARE COMPULSORY**.
(There are 4 questions in total.
The maximum total score is 100.)

Family name (in block letters): .....
Given name: .....
Student number: .....

INSTRUCTIONS

You are required to complete this examination by writing your answers into this booklet. Show your working as you complete this examination booklet, so that you can more easily receive partial credit.

If you need more space for your answers, several extra pages can be found at the end of this booklet. Make a note indicating when your work is continued on an extra page.

This page is intentionally blank

Exam starts on Page 4

Q1. **(a)** Logic functions and design:

A	B	C	S
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

(i) **[2 marks]**

From the above truth table, write a Boolean expression for S in "sum-of-minterms" form (i.e., each term in the expression should contain each of the input symbols).

(ii) **[2 marks]**

Using the rules of Boolean algebra, show how the "sum-of-minterms" expression for S can be simplified to obtain the Boolean expression

$$S = \overline{A + B} + BC.$$

(iii) [3 marks]

Sketch an implementation of the Boolean expression $S = \overline{A + B} + BC$, using only AND, OR, and NOT logic gates.

(iv) [1 mark]

You notice that the first term of the function for S is the `NOR` function. Is it possible to represent the entire function using only `NOR` expressions, and therefore implement an equivalent logic circuit using only `NOR` gates?

(b) You are designing the control aspect of a CPU, and you know that you need all of the subcomponents shown in Figure 1.

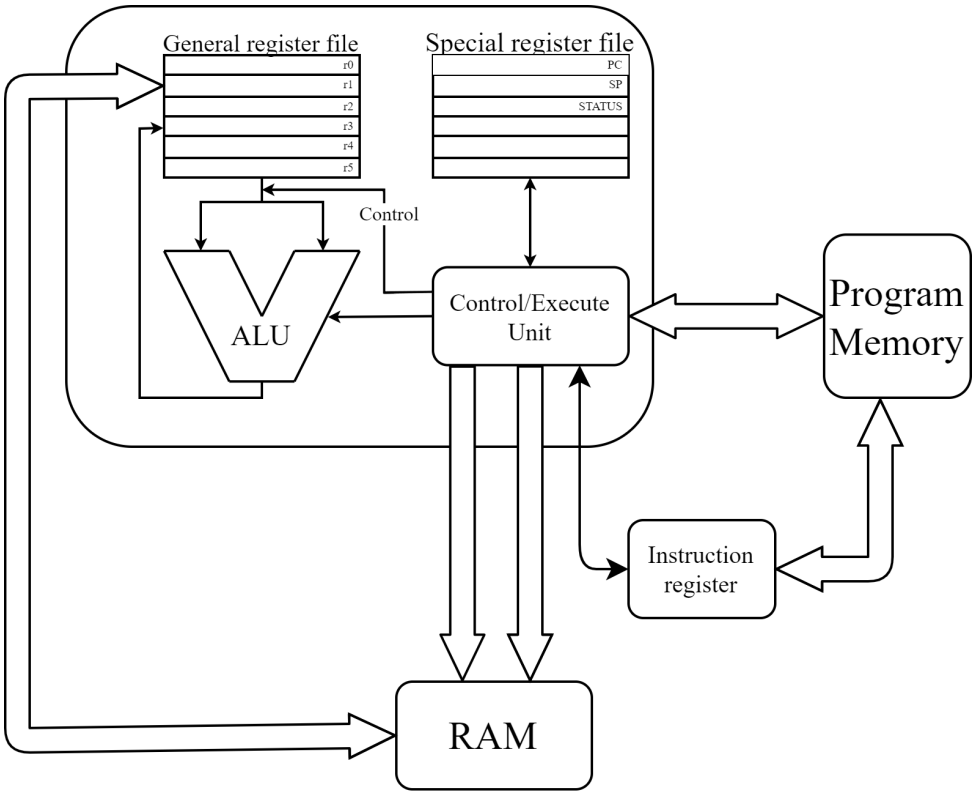


Figure 1: Block diagram of a computer architecture.

(i) [3 marks]

Draw **two** arrows on Figure 1, one showing the total path an instruction takes, **AND** one showing the path data takes to perform an operation, and store the result from this to memory. Label your arrows “I" for instruction and “D" for data.

(ii) [2 marks]

The architecture shown in Figure 1 is Harvard, where the program memory is separate from the data memory. State two advantages of such a configuration.

The ATMega32u2 is a good example to use to learn about computer architecture. Its block diagram is shown in Figure 2.

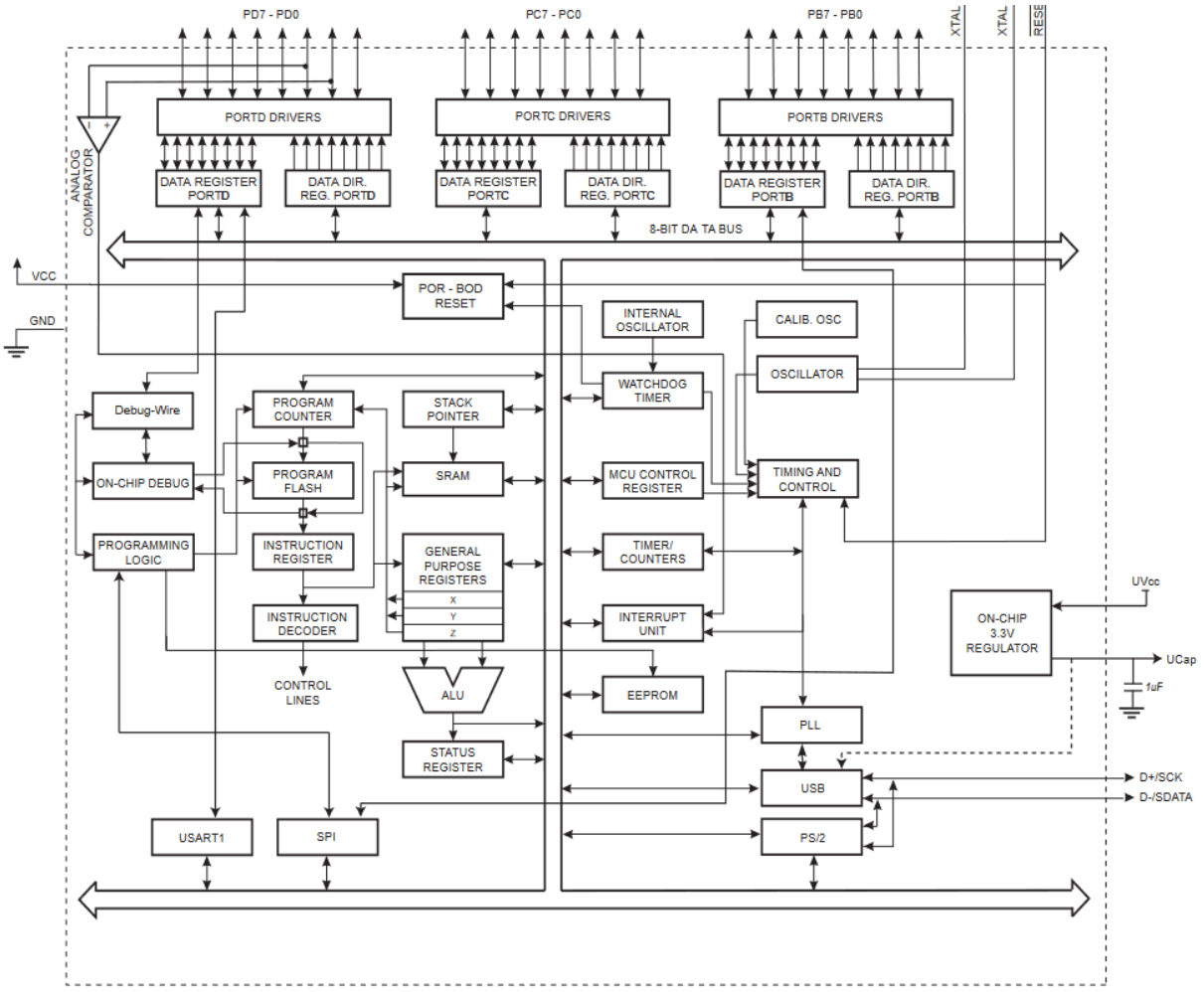


Figure 2: Block diagram of the ATMega32u2.

(iii) [4 marks]

On the diagram in Figure 2, label and identify the components A) through F).

A) Somewhere a switch would be connected to.

B) Somewhere the direct result of an arithmetic operation would be stored.

C) A peripheral used to count the number of times a button is pushed.

D) Somewhere a large array of variables would be stored

E) The control and execute unit.

F) The pointer to the next instruction to be run.

(c) Table 1 is the truth table for an individual Full Adder circuit which performs addition on 1-bit numbers. The 4-bit Ripple-Carry Adder shown in Figure 3 uses four Full Adder circuits to perform addition on 4-bit binary numbers.

Table 1: Truth table for a Full Adder.

Inputs			Outputs	
A	B	C_{in} (carry in)	S (sum)	C_{out} (carry out)
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

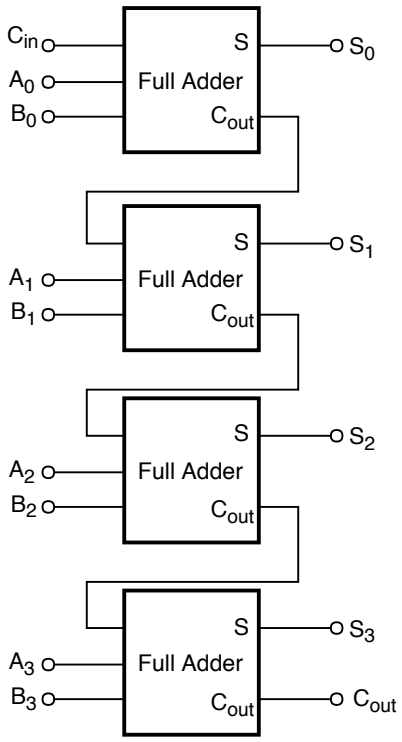


Figure 3: 4-bit Ripple-Carry Adder for context.

Consider now an 8-bit ripple counter, as would be found in an 8-bit ALU.

(i) [1 mark]

The Ripple-Carry Adder is to be used to analyse the base-10 numbers A = 13 and B = 16. Write the 8-bit binary sequences, $A_7A_6A_5A_4A_3A_2A_1A_0$ and $B_7B_6B_5B_4B_3B_2B_1B_0$, that represent the inputs to the Ripple-Carry Adder.

(ii) [2 marks]

You wish to perform the operation $A - B$. State the two required changes necessary to change from a system configured to perform $A + B$ to $A - B$ within an ALU.

(iii) [3 marks]

Give the binary values of the operands an ALU would use when calculating $A - B$, **AND** calculate the result.

(iv) [2 marks]

Interpret you answer from **(iii)** in two's complement, **AND** give two other interpretations of the binary sequence.

[If you have no answer from above use `0b10100011`.]

Q2. **(a)** Memory is one of the critical parts of any computer device. One of the base units of memory is the flip-flop, for example that shown in Figure 4.

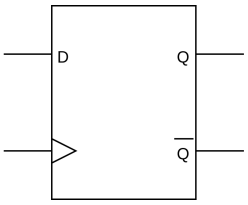


Figure 4: A flip-flop

(i) **[2 marks]**

State the two functions that memory must perform, and briefly discuss how a flip-flop achieves these required functions.

An example of a sequential logic circuit is shown in Figure 5.

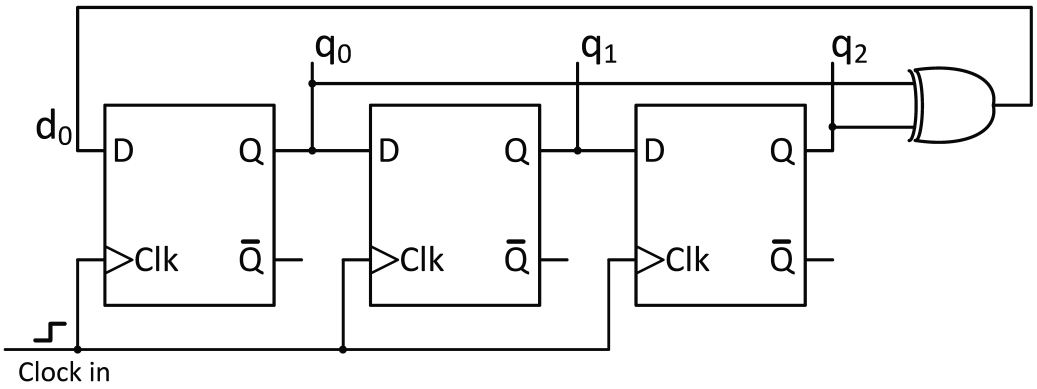


Figure 5: An example 4-bit sequential logic circuit.

(ii) [5 marks]

For the sequential logic circuit shown in Figure 5, determine the next five states of q_0, q_1, and q_2. Assume that the starting sequence $q_0q_1q_2$ is 111.

(b) Programmable GPIO

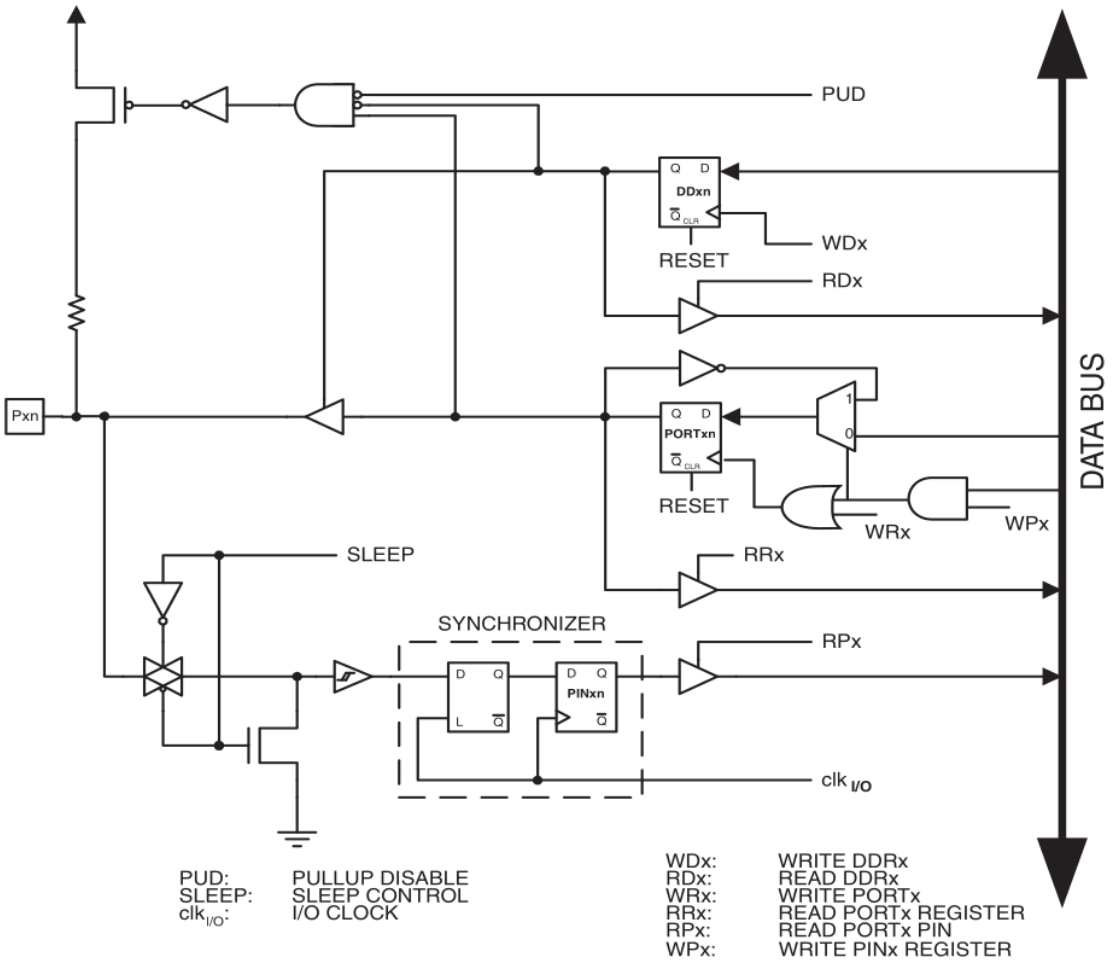


Figure 6: Diagram of the internal GPIO circuitry of the ATMega32u2.

(i) [4 marks]

Explain how a GPIO pin such as `Pxn` in Figure 6 is able to act as an input with the internal pull up resistor enabled. In your answer refer to the critical components shown in Figure 6 **AND** state the values of key registers required for this functionality.

You want to implement a variable which increments (counts up) at a constant rate. You know that you can either introduce a variable (such as a `uint8_t`) in software, which you increment each time through your main loop, or you can use the built-in timer/counter peripheral (in hardware) configured to count upwards.

(ii) [3 marks]

State which is the more robust solution, and justify your answer.

(iii) [2 marks]

You plan to use your counter value to generate a pulse-width modulation signal. Briefly discuss what a pulse width modulation signal is, and why they are used.

(c) An AVR instruction for the ATMega32u2 is shown in Table 2.

Table 2: Operation specification for the `call` instruction.

Syntax	Operands	Operation
`call k`	$0 \leq k \leq 64K$	STACK ← PC + 2
Control registers	Opcode	Flags
PC ← k SP ← SP - 2	1001 010k kkkk 111k kkkk kkkk kkkk kkkk	None

(i) [1 mark]

What impact does the `call` instruction achieve through manipulation of the programme counter?

(ii) [2 marks]

The operation of the `call` instruction is to put the value of PC + 2 on to the stack. What is the purpose of storing the programme counter, and why does it store PC + 2 specifically?

(iii) [2 marks]

State three other pieces of information that are stored on the stack to form the programme context / stack frame.

(iv) [3 marks]

The status register stores several flags from the ALU. Briefly explain what the purpose of flags is, and give one example of how they are used.

(d) [1 mark]

How is every piece of information within computers stored?

This page is intentionally blank

Q3. Embedded Systems 1

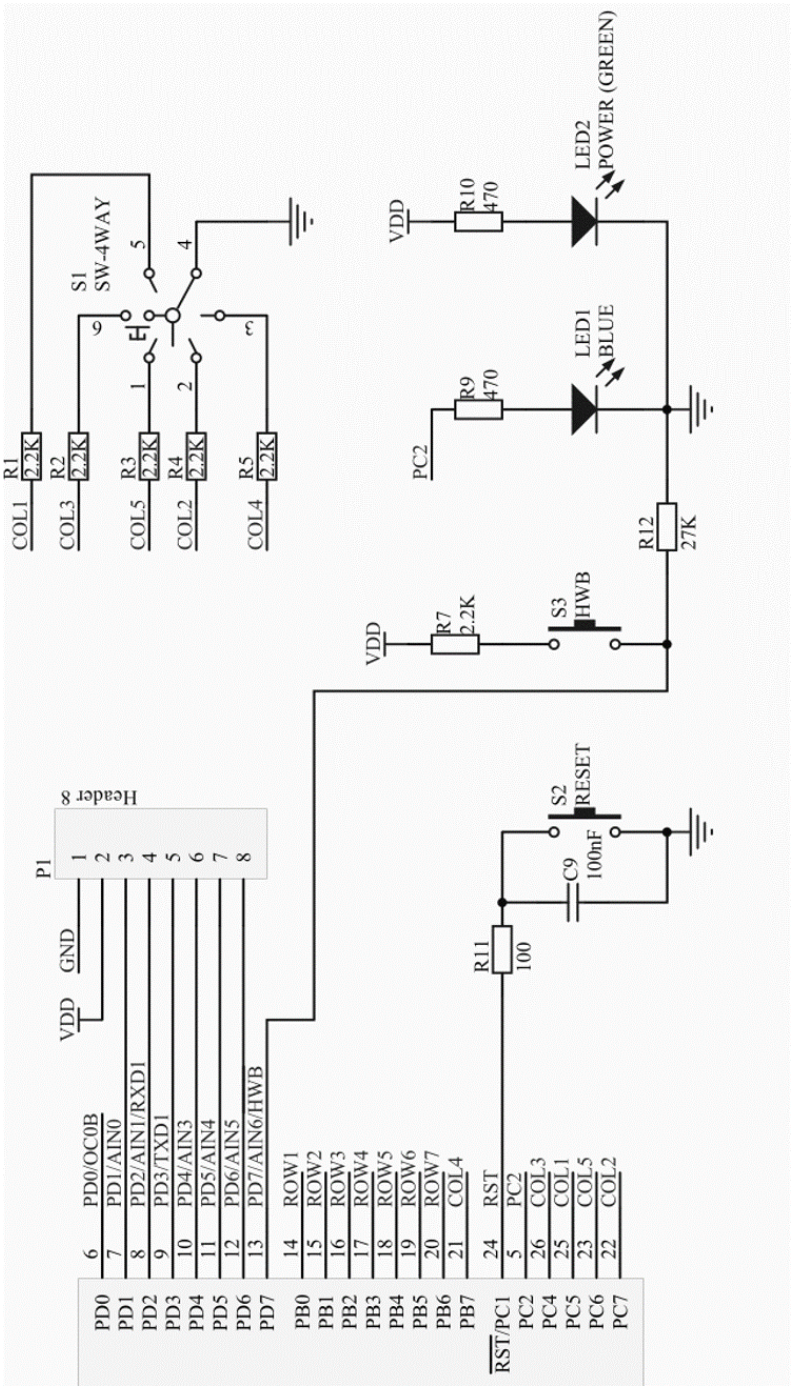


Figure 7: Schematic showing the GPIO pins of the ATmega32u2 microcontroller of the UCFK4 for Q3(a).

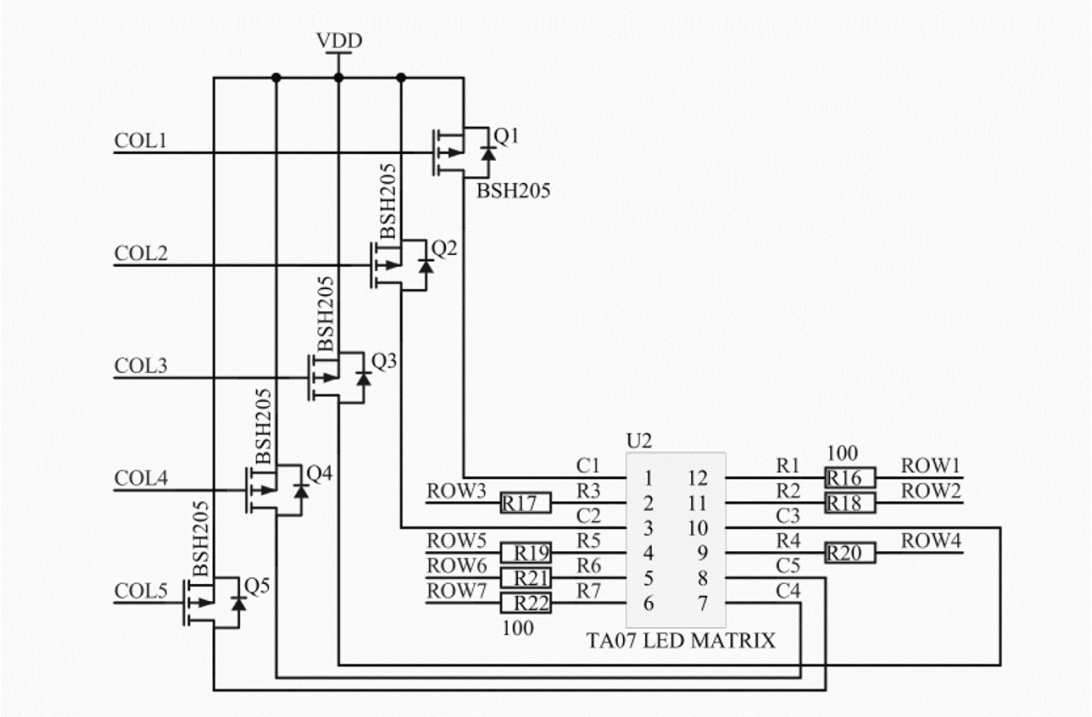


Figure 8: Schematic showing the LED matrix display of the UCFK4 for Q3(a).

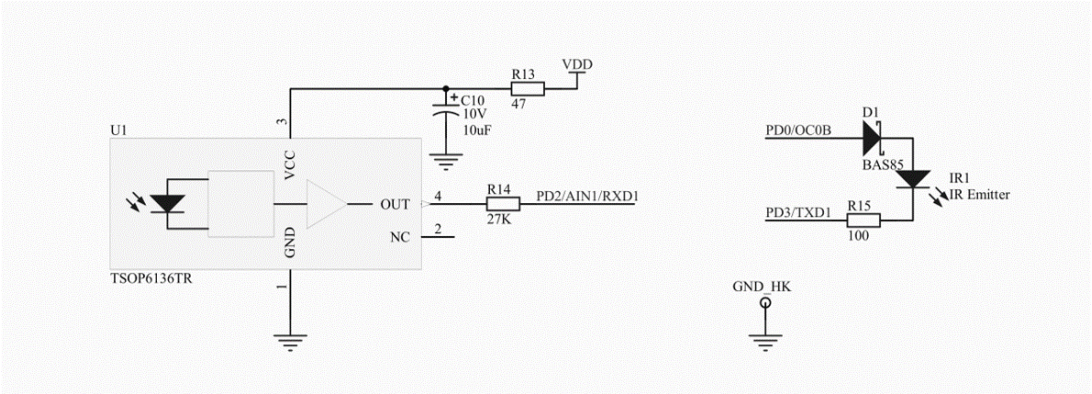


Figure 9: Schematic showing the IR communications of the UCFK4 for Q3(a).

(a) The circuit schematic for the UCFK4 is shown in Figures 7, 8 and 9. Answer the following questions using these schematics.

(i) [2 marks]

What is the purpose of resistor R7?

(ii) [2 marks]

Assuming that S3 has been set as an input, write an `if` conditional statement in C that evaluates to `True` when the switch S3 is pushed.

(iii) [2 marks]

Assuming that LED1 (Blue LED) has been set as an output, write a statement in C that toggles the state of LED1 but does not affect any of the other outputs on the UCFK4.

(iv) [2 marks]

The UCFK4 is able to produce sound by connecting a piezo buzzer across two GPIO pins of the microcontroller. Identify two pins of the microcontroller that can be used for the piezo buzzer. Justify your selection.

(v) [2 marks]

Write a single statement in C that sets the Data Direction Register for the two pins chosen in Q3(a)(iv), and does not affect any of the other inputs or outputs in that port.

(vi) [2 marks]

Why are the circuit elements Q1 to Q5 required?

(vii) [1 mark]

Why is PB7 (COL4) connected to both the navswitch (S1) and the LED matrix display?

(viii) [2 marks]

Describe the software required on the microcontroller for PB7 (COL4) to be connected to both the navswitch (S1) and the LED matrix display.

(b) [4 marks]

Data is transmitted serially from one UART to another. Assume the UARTs have been configured for 7-bit data, the LSB is transmitted first, the idle state is high, and there is one stop bit, one start bit, and an odd parity bit. Sketch the waveform that should be transmitted from the UART to transmit the ASCII character #. An ASCII table is shown in Table 3.

Table 3: The ASCII character table for Q3(b).

0	NUL	32		64	@	96	‘
1	^A	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	^D	36	$	68	D	100	d
5	^E	37	%	69	E	101	e
6	^F	38	&	70	F	102	f
7	BELL	39	’	71	G	103	g
8	BS	40	(	72	H	104	h
9	TAB	41	)	73	I	105	i
10	LF	42	*	74	J	106	z
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	^N	46	.	78	N	110	n
15	^O	47	/	79	O	111	o
16	^P	48	0	80	P	112	p
17	XON	49	1	81	Q	113	q
18	^R	50	2	82	R	114	r
19	XOFF	51	3	83	S	115	s
20	^T	52	4	84	T	116	t
21	^U	53	5	85	U	117	u
22	^V	54	6	86	V	118	v
23	^W	55	7	87	W	119	w
24	^X	56	8	88	X	120	x
25	^Y	57	9	89	Y	121	y
26	^Z	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	|
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

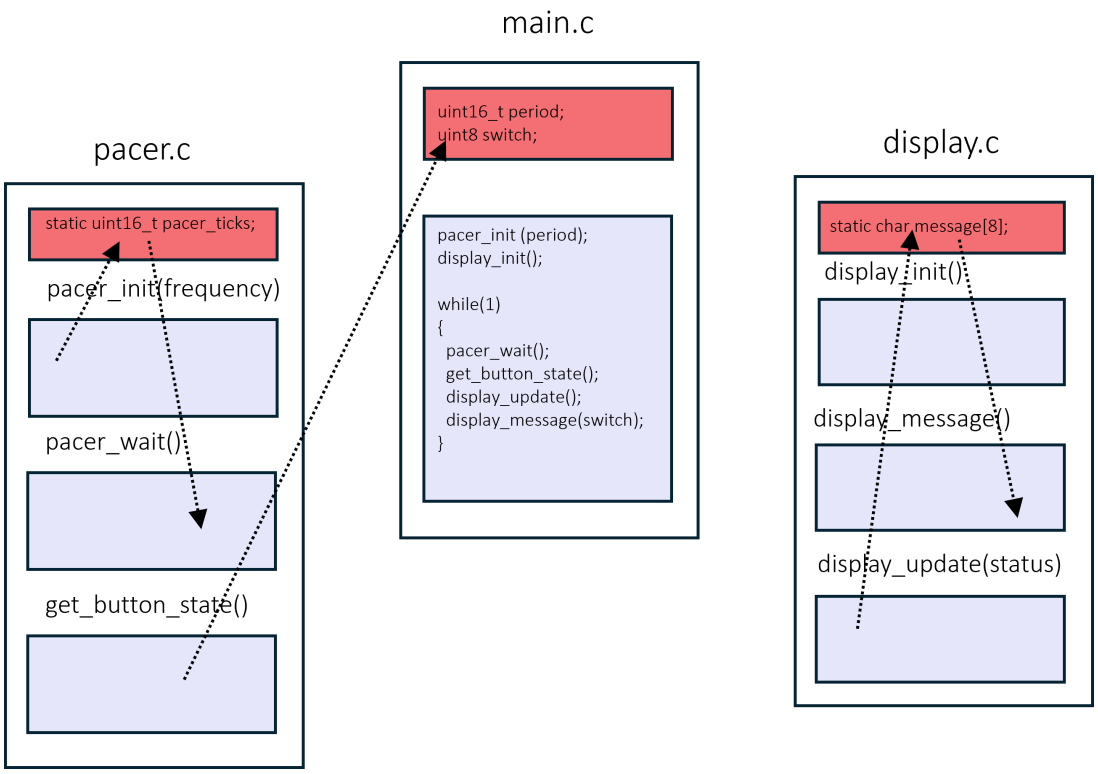


Figure 10: The software structure diagram for Q4(c).

(c) A software structure diagram for a program on the UCFK4 is shown in Figure 10. The dotted arrows indicate the use or assignment of a variable. Answer the following questions.

(i) **[3 marks]**

Briefly define *coupling* in embedded systems programs, and comment on the coupling between modules in Figure 10.

(ii) [3 marks]

Briefly define *cohesion* in embedded systems programs, and comment on the cohesion of the modules in Figure 10.

Q4. Embedded Systems 2

Listing 1: Source listing for Q4(a)

```
#include "system.h"
#include "button.h"
#include "timer.h"
#include "display.h"

#define CPU_F 8000000          /* Hz */
#define BUTTON_RATE 50         /* Hz */
#define DISPLAY_RATE 75        /* Hz */
#define TIMER_RATE 100         /* Hz */
#define PACER_RATE             /* Hz */

static uint16_t pacer_ticks;

void pacey_wait(void) {
   while (TCNT1 < pacer_ticks)
      continue;
   TCNT1 = 0;
 }

void pacey_init(uint16_t pacer_frequency){
    TCCR1B = 0x03;
    pacer_ticks =
    }

int main (void){

    system_init();
    display_init();
    button_init();
    timer_init()
    pacey_init(PACER_RATE);

    /*Complete so tasks run at required frequencies */

    return 0;
}
```

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Clock Select Bit Description

CSn2	CSn1	CSn0	Description
0	0	0	No clock source. (Timer/Counter stopped)
0	0	1	$clk_{I/O}$/1 (No prescaling
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on Tn pin. Clock on falling edge
1	1	1	External clock source on Tn pin. Clock on rising edge

Figure 11: TCCR1B register bits for Q4(a).

Bit	7	6	5	4	3	2	1	0	
(0x85)	TCNT1[15:8]								TCNT1H
(0x84)	TCNT1[7:0]								TCNT1L
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figure 12: TCNT1 register bits for Q4(a).

(a) Romeo and Luna have started to write the C program `countdown.c` in Listing 1 for the ATmega32u2 microcontroller. The TCCR1B configuration register is shown in Figure 11, and the TCNT1 register in Figure 12. `countdown.c` has three tasks that need to be completed:
`void button_task(void)` at 50 Hz
`void display_task(void)` at 75 Hz
`void timer_task(void)` at 100 Hz.
Given a CPU frequency of 8 MHz, answer the following.

(i) [1 mark]

What is the **decimal** value of the prescaler in Listing 1?

(ii) [2 marks]

What is the maximum period of the timer in Listing 1?

(iii) [2 marks]

What is the minimum value of `PACER_RATE` that can perform the three tasks at the specified frequencies? Justify your answer.

(iv) [2 marks]

Complete Line 22 of Listing 1 in the following box.

```
pacer_ticks =
```

(v) [6 marks]

In the following box, complete Listing 1 from Line 33 onwards so that the three task functions are called at their specified frequencies.

```
    /*Complete so tasks run at required frequencies */




    return 0;
```

(vi) [2 marks]

The variable `pacer_ticks` on Line 12 of Listing 1 is designated with the keyword `static`. Give two attributes of `static` variables in embedded systems C programming.

(vii) [1 mark]

The version of `countdown.c` in Listing 1 before you made any changes is in an existing git repository, which has been initialized in Romeo's current directory and connected to a remote server. Romeo makes your suggested changes to `countdown.c` in his local directory. In the following box, write the git command(s) to see your changes made to `countdown.c`.

(viii) [2 marks]

Romeo decides that he wishes to update the existing git repository, which has been initialized in his current directory and connected to a remote server. In the following box, write the git command(s) to ensure `countdown.c` will be up-to-date in the remote repository.

(ix) [1 mark]

Luna is also a member of the remote git repository, which has been initialized in her current directory and is connected to a remote server. Using the following box, write the git command(s) so that Luna's local repository is up-to-date.

(b) The file dependency graph for the program `hello1` is shown in Figure 13. Note that all `.c` files also depend on `system.h` but these dependencies are not shown in Figure 13 for clarity. The first version of the `makefile` for this file dependency graph is given in Listing 2.

(i) **[6 marks]**

The program `hello1` does not compile and load on the microcontroller using the `makefile` in Listing 2 with `make program`. Assuming the file dependency graph in Figure 13 is correct, identify and list six (and not more than six) errors in Listing 2.

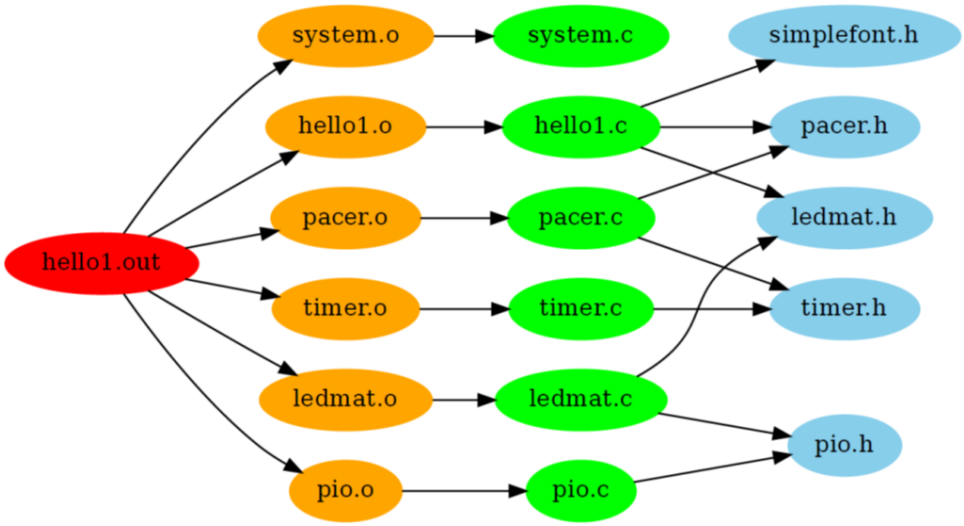


Figure 13: File dependency graph for Q4(b).

Listing 2: Makefile for Question 4(b).

```
CC = avr-gcc
CFLAGS = -mmcu=atmega32u2 -Os -Wall

DEL = rm

all: hello1.out

hello1.o: hello1.c system.h pacer.h ledmat.h simplefont.h
	$(CC) -c $(CFLAGS) $< -o $@

system.o: system.c system.h
	$(CC) -c $(CFLAGS) $< -o $@

timer.o: timer.c system.h timer.h
	$(CC) -c $(CFLAGS) $< -o $@

ledmat.o: ledmat.c pio.h system.h ledmat.h
	$(CC) -c $(CFLAGS) $< -o $@

pacer.o: pacer.c system.h pacer.h
	$(CC) -c $(CFLAGS) $< -o $@

hello1.out: hello1.o pio.o system.o timer.o ledmat.o
	$(CC) $(CFLAGS) $< -o $@ -lm

hello1.hex: hello1.out
	$(OBJCOPY) -O ihex hello1.out hello1.hex

.PHONY: clean
clean:
	-$(DEL) *.o *.out *.hex

.PHONY: program
program: hello1.out
	dfu-programmer atmega32u2 erase; dfu-programmer \
	    atmega32u2 flash hello1.hex; \
	        dfu-programmer atmega32u2 start
```

END OF QUESTIONS

Extra answer space (optional)

Extra answer space (optional)

END OF EXAM